

Logistic Regression

Credits: Dr. Parul
Arora

Contents to be covered

- Logistic Regression
- Perceptron
- Newton's method

Probabilistic Discriminative model: Logistic Regression

- The goal of binary logistic regression is to train a classifier that can make a binary **decision about the class of a new input observation**.
- It focus on modeling the decision boundary between classes in a classification problem.
- **Maximum likelihood estimation** is often used to estimate the parameters of the discriminative model, such as the coefficients of a logistic regression model

Logistic regression - Introduction

- The linear regression is not suitable for classification problem. Linear regression is unbounded, and this brings logistic regression.
- Logistic regression measures
 - the relationship between the dependent variable (categorical) and one or more independent variables by **estimating probabilities** using a logistic function.
- On the mathematical side, the logistic regression model will pass the likelihood occurrences through the **logistic function** to predict the corresponding target class. This popular generalized logistic function is the ***Softmax function***.
- Logistic Regression is a popular statistical model used for binary classification, that is for predictions of the type this or that, yes or no, A or B, etc. Can be used for multiclass classification as well
- Example, consider the task of predicting someone's gender (Male/Female) based on their Weight and Height

Logistic Regression

- **Logistic regression** estimates the probability of a certain event occurring using the odds ratio by calculating the logarithm of the odds
- Uses **Maximum likelihood estimation** (MLE) to transform the probability of an event occurring into its odds, a nonlinear model
- Odds ratio is the probability of occurrence of a particular event over the probability of non occurrence
- Odds ratio is useful in providing an estimate of the magnitude of the relationship between binary variables
- Allows one to examine the effects of the variables on the relationship- how y varies when x varies

Logistic Regression : Classification Algorithm

Binary classification $y \in \{0,1\}$

$$h(x) = \sum_{i=0}^n \theta_i x_i \quad \text{by considering , } x_0 = 1$$

n = number of features

$h_{\theta}(x) \in [0,1]$

$$= \theta^T x$$

To obtain a logistic regression, we apply an activation function known as sigmoid function to this linear hypothesis,

$$h_{\theta}(x) = \sigma(\theta^T x)$$

where

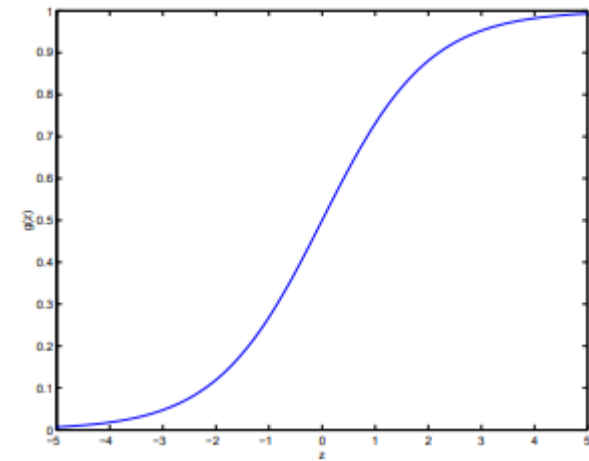
$$z = \theta^T x$$

$g(z)$ is thus our logistic regression function

and is defined
$$g(z) = \frac{1}{1 + e^{-z}}$$

Hence;

$$h_{\theta}(x) = \sigma(z) = g(z)$$



$g(z)$ tends towards 1 as $z \rightarrow \infty$, and $g(z)$ tends towards 0 as $z \rightarrow -\infty$.

As it maps very large input domain to a small range of inputs, it is often referred as *squashing function*.

Given the logistic regression model, how do we fit θ for it?

As we saw least squares regression could be derived as the maximum likelihood estimator under a set of assumptions,

Endow the classification model with a set of probabilistic assumptions, and then fit the parameters via maximum likelihood.

Let us assume that

$$\begin{aligned}P(y = 1 \mid x; \theta) &= h_{\theta}(x) \\P(y = 0 \mid x; \theta) &= 1 - h_{\theta}(x)\end{aligned}$$

Note that this can be written more compactly as

$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

- Maximum Likelihood Estimation (MLE) is a statistical method for training machine learning models by finding parameter values that maximize the likelihood of observing the given training data.
- It calculates the probability distribution parameters (like mean and variance) that make the observed data most probable.

$$L(\theta) = p(\vec{y}|X; \theta)$$

$$= \prod_{i=1}^m p(y^{(i)}|x^{(i)}; \theta)$$

$$= \prod_{i=1}^m \left(h_{\theta}(x^{(i)}) \right)^{y^{(i)}} \left(1 - h_{\theta}(x^{(i)}) \right)^{1-y^{(i)}}$$

$$l(\theta) = \ln L(\theta)$$

Log Likelihood

$$= \sum_{i=1}^m y^{(i)} \ln h(x^{(i)}) + (1 - y^{(i)}) \ln(1 - h(x^{(i)}))$$

$$\theta := \theta + \alpha \nabla_{\theta} \ell(\theta).$$

Let's start by working with just one training example (x, y) , and take derivatives to derive the stochastic gradient ascent rule:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= (y - h_{\theta}(x)) x_j \end{aligned}$$

$$\begin{aligned} g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\ &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\ &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})} \right) \\ &= g(z)(1 - g(z)). \end{aligned}$$

$$g'(z) = g(z)(1 - g(z))$$

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Example 1

(Logistic Regression Classifier)

- Consider the logistic regression model that assigns to the point (x_1, x_2) the prediction

$$\hat{y} = \sigma(2x_1 + 3x_2 - 4)$$

and the point $s = (1,1)$ with label 0.

- Calculate the prediction that the model gives to the point p .
- Calculate the log-loss or error that the model produces at the point p .
- Use the logistic regression to obtain a new model which produces a smaller log-loss or error. You can use 0.1 as the learning rate.

Example 1 (Logistic Regression Classifier)

Solution:

a) The prediction is

$$p(C_1 | \mathbf{x}) = \sigma(\theta^T \mathbf{x}) = y(\mathbf{x})$$

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

$$\begin{aligned}\sigma(a) &= \frac{1}{1 + \exp(-(2(1) + 3(1) - 4))} \\ &= 0.731\end{aligned}$$

Example 1 (Logistic Regression Classifier)

Solution:

b) The log loss is

$$E(\mathbf{w}) = -\ln L(\mathbf{w}) = -\ln \prod_{n=1}^N P(C_1 | \mathbf{x}_n)^{t_n} (1 - P(C_1 | \mathbf{x}_n))^{1-t_n}$$

For label "0", $t_n=0$

$$\begin{aligned} E(\mathbf{w}) &= -\ln(1-0.731) \\ &= 1.313 \end{aligned}$$

If label "1", $t_n=1$

$$\begin{aligned} \text{Log loss} &= E(\mathbf{w}) = -\ln(0.731) \\ &= 0.3133 \end{aligned}$$

Example 1 (Logistic Regression Classifier)

Solution:

c) Recall that the perceptron trick for the logistic regression model with prediction gives us the new weights

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} + \eta(t_n - y_n)\mathbf{x}$$

$$w_0^{\text{new}} = w_0^{\text{old}} + \eta(t_n - y_n)$$

$$\theta^1 = \begin{bmatrix} \mathbf{w}_1 \\ \mathbf{w}_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y)\mathbf{x}_1 \\ (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y)\mathbf{x}_2 \\ \sigma(\mathbf{w} \cdot \mathbf{x} + b) - y \end{bmatrix}$$

Example 1 (Logistic Regression Classifier)

- Substituting the values
- We get that the new weights for our classifier as 1.9269, 2.9269, -4.07319 with label 0.

Thus, our new classifier is the one that makes the prediction

$$\hat{y} = \sigma(1.9269x_1 + 2.9269x_2 - 4.07319)$$

Now, updated prediction and error

$$\hat{y} = \sigma(0.781) = 0.686$$

$$\log loss = -\ln(1 - 0.686) = 1.15$$

Example 2 (Logistic Regression Classifier)

- Using the first model in example 1, construct a point for which the prediction is 0.8.
- HINT: First find the score that will give a prediction of 0.8, and recall that the prediction
- Let score= a , prediction= b

$$\hat{y} = \sigma(a) = b$$

$$\frac{1}{1 + \exp(-a)} = b$$

$$a = \ln\left(\frac{b}{1-b}\right)$$

$$a = \ln(4) = 1.386$$

Example 2 (Logistic Regression Classifier)

Recall that for the point (x_1, x_2) , the score is

$$2x_1 + 3x_2 - 4 = 1.386$$

Many points satisfy that the score is 1.386, but in particular, let's pick one in which $x_2 = 0$.

Example 2 (Logistic Regression Classifier)

Solution (cont.)

Solve the equation:

$$2x_1 + 3 \cdot 0 - 4 = 1.386$$

which has solution

$$x_1 = 2.693$$

Thus, a point that gives a prediction of 0.8 is the point (2.693, 0).

Softmax Function

- For case of $K > 2$ classes, we have the following multi-class generalization (**Softmax regression**):

$$\begin{aligned} p(\mathcal{C}_k | \mathbf{x}) &= \frac{p(\mathbf{x} | \mathcal{C}_k) p(\mathcal{C}_k)}{\sum_j p(\mathbf{x} | \mathcal{C}_j) p(\mathcal{C}_j)} \\ &= \frac{\exp(a_k)}{\sum_j \exp(a_j)} \end{aligned}$$

where $a_k = \ln (p(\mathbf{x} | \mathcal{C}_k) p(\mathcal{C}_k))$

- The normalized exponential is also known as the **softmax function**.
- We now look at some specific forms of class conditional distributions, depending upon whether input variables are continuous or discrete.

Logistic regression vs. Softmax regression

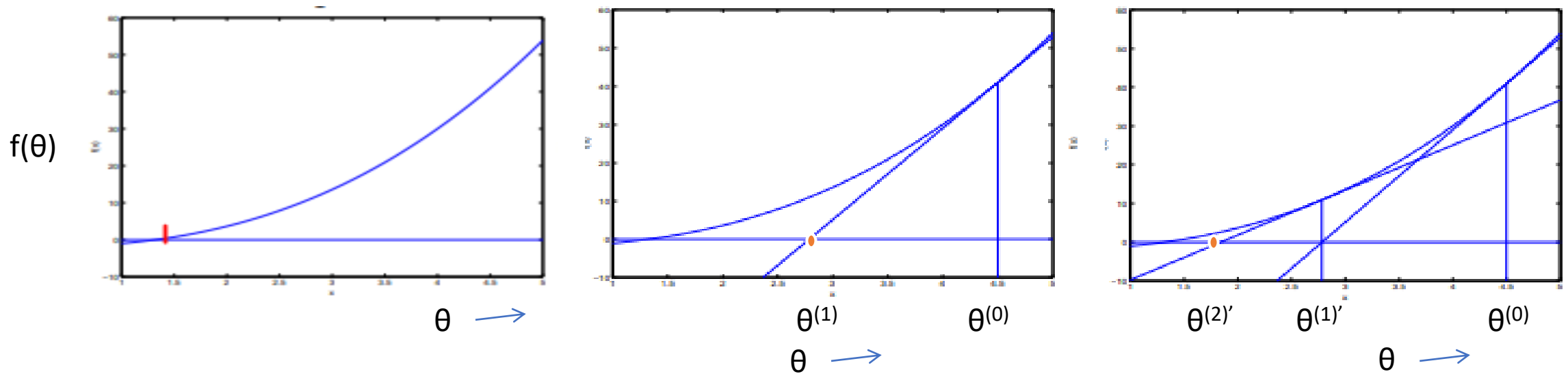
Feature	Logistic Regression	Softmax Regression
Classes	2 (Binary)	More than 2 (Multi-class)
Activation Function	Sigmoid = $\frac{1}{1 + e^{-z}}$	Softmax $\frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$

Newton's Method

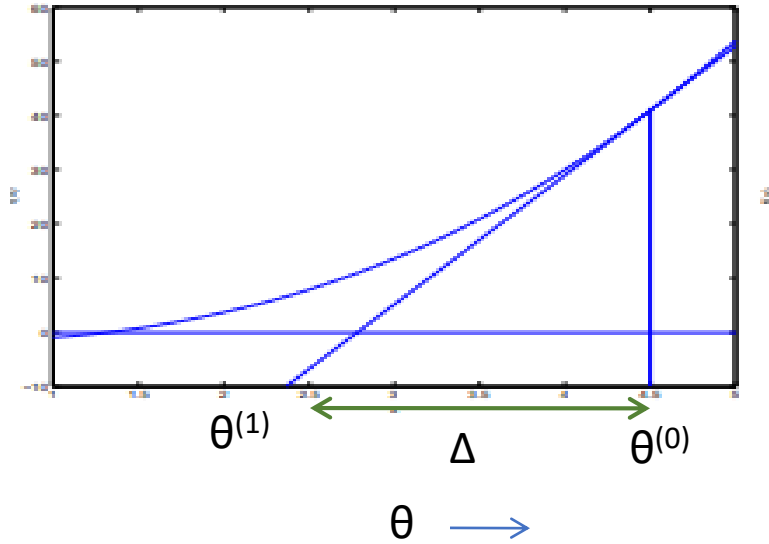
- It is used to find a zero of a function.

$f : \mathbb{R} \rightarrow \mathbb{R}$

Value of θ , $f(\theta) = 0$



Newton's method gives a way of getting to $f(\theta) = 0$.



$$f'(\theta^{(0)}) = \frac{f(\theta^{(0)})}{\Delta}$$

$$\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$$

$$\theta^{(1)} = \theta^{(0)} - \Delta$$

$$\theta^{(1)} = \theta^{(0)} - \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$$

if we want to maximise loglikelihood function $l(\theta)$,
 The maxima of ℓ correspond to points
 where its first derivative $\ell'(\theta)$ is zero. $l'(\theta)=0$

Updation Rule for Newton's method

For scalar θ

$$\theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})} \quad \text{one iteration of Newton's Method.}$$

The maxima of $\ell(\theta)$ correspond to points where its first derivative $\ell'(\theta)$ is zero.

So, by letting $f(\theta) = \ell'(\theta) = 0$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\ell'(\theta^{(t)})}{\ell''(\theta^{(t)})}$$

For vector θ

$$\theta^{(t+1)} = \theta^{(t)} - H^{-1} \Delta_{\theta} \ell(\theta)$$

Newton's Raphson method

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}$$

Hessian Matrix

Newton's method typically enjoys faster convergence than (batch) gradient descent, and requires many fewer iterations to get very close to the minimum. However it can be more expensive to work with Hessian matrix.